KWAME NKRUMAH UNIVERSITY OF SCIENCE AND TECHNOLOGY

ELECTRICAL/ ELECTRONIC ENGINEERING

EE 371

MICROPROCESSORS LAB REPORT

GROUP 14

GROUP MEMBERS

Deveer Benedict Nii - 7322323
Cornan John - 7321223
Boateng Lord Nkansah - 7320823
Manteaw Emmanuel - 7328823
Ofosu Maame Akua Henewaa - 7332023
Cudjoe Hermone - 7321423
Buabin Michael - 7321023
Issah Touffic - 7326223
Maisie Pius Kweku Emmanuel - 7328723

TABLE OF CONTENT

- Introduction
- Problem Statement
- Methodology
- Components used and their function
- Code Overview
- Circuit Schematic
- Demonstration
- Future Roadmap: Scalability and Innovation
- Challenges encountered
- Conclusion

INTRODUCTION

This report outlines the development, design, and testing of a proposed Arduino-based Emergency Alert System for patient care settings. The proposed system allows patients to press a special call button, which will instantly alert the nursing staff through a combination of a message displayed on the LCD screen, a sound alarm from the buzzer, and a visual indicator from the LED.

The proposed project was carried out as part of a course on Engineering Design, where the entire development cycle, from problem identification to hardware selection, circuit development, software coding, circuit simulation using Tinkercad, and implementation of the proposed Arduino-based solution, is covered.

Problem Statement

Background

In hospitals, clinics, and other health facilities, emergencies frequently arise when patients are alone in their rooms, unable to physically reach a nurse. The existing nurse call systems can be unreliable, hard to use, and do not give a visual cue as to which patient is calling for emergency care. This leads to life-threatening delays.

Identified Challenges

•Patients who are unable to move cannot verbally contact a nurse during emergencies.

•Nurses who are busy attending other patients may not hear a patient's verbal call for emergency care from another patient's room.

•With the existing basic nurse call systems, there is no visual cue displayed showing which patient requires emergency care.

•There is no effective management of multiple emergency calls from other patients.

•Once a patient's emergency call is generated, there is no provision for clearing the emergency call and queuing other emergency calls.

Objective

The objective of this project was to design and develop a reliable emergency patient alarm system using an Arduino Uno microcontroller, meeting the following specifications:

•Each patient can press a special button to send emergency alerts displayed on the LCD screen.

•Once a patient's emergency

METHODOLOGY

1. Identify Need

We analyzed the scenarios for patient emergencies. We recognized a lack of communication with manual nurse call systems, prompting us to concentrate on a reliable alarm notification system.

2. Design System

We moved on to designing our system. In this step, we decided on our hardware stack. We decided on using Arduino Uno for processing, a 16x2 LCD for display, and a combination of push buttons, buzzers, and LEDs for user interface.

3. Build & Wire

We moved on to building our system. We used a breadboard to connect our components. We mapped the circuit diagram and verified every pin connection to ensure a stable hardware platform.

4. Write Code

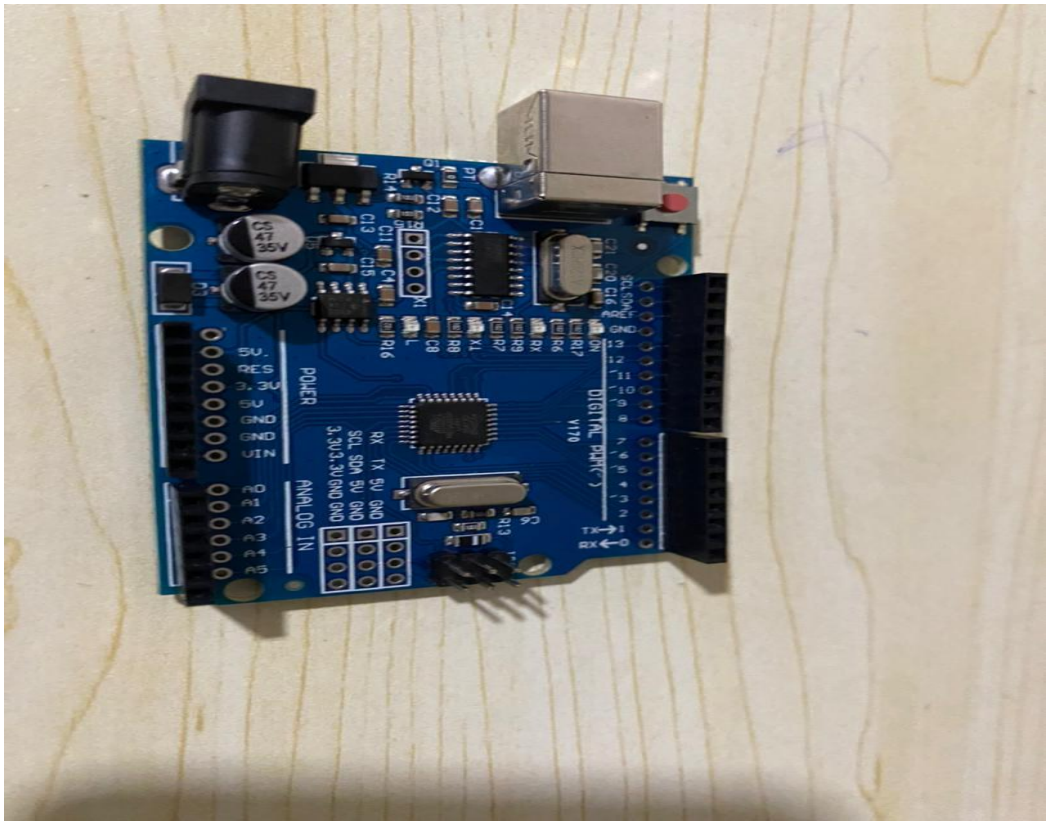
We moved on to writing our code. We used C++ for our implementation. Some of the features we implemented include a notification system for multiple patients, a 2-minute auto-expiry for notifications, and a non-blocking implementation for buzzers using the millis() function.

5. Test & Refine

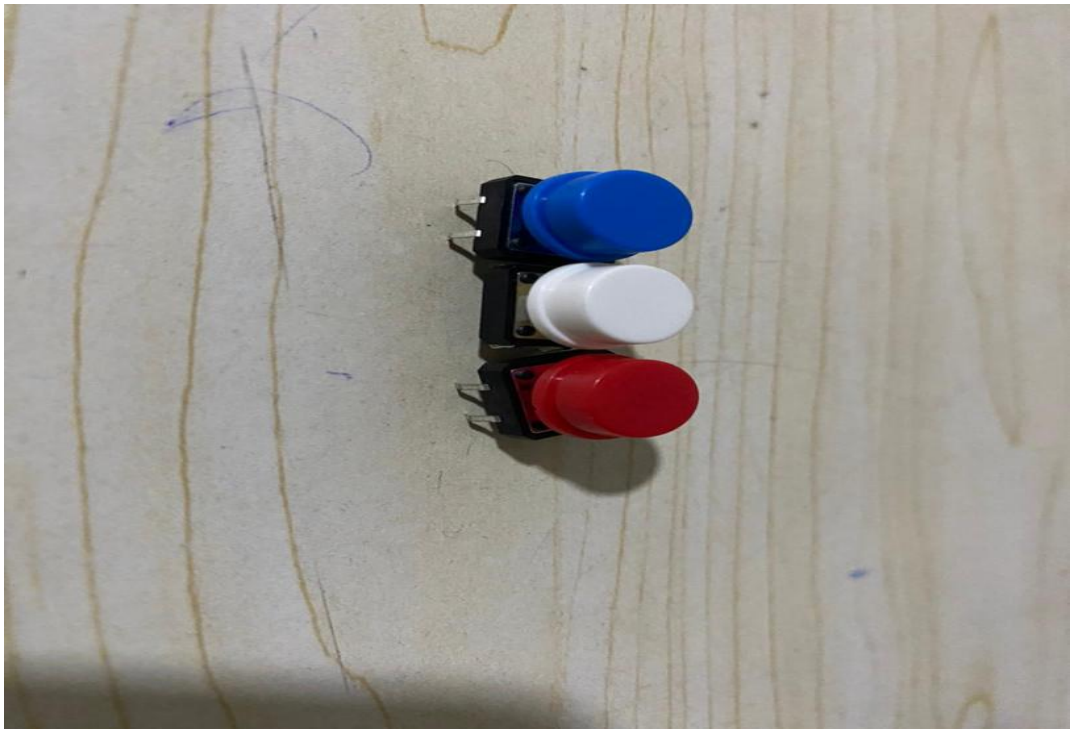
We moved on to the final step—testing. We simulated our code using Tinkercad and then moved on to the physical Arduino. This step was crucial in resolving LCD display issues.

Components used and their Functions

Arduino Uno Mini: This is the "brain" of your project. This is essentially the same thing as the classic Arduino Uno, but it's smaller, made especially for collectors, and has a USB-C connector. It does exactly the same thing, though. It runs your program, reads the buttons, and sends information to the screen and the buzzer.



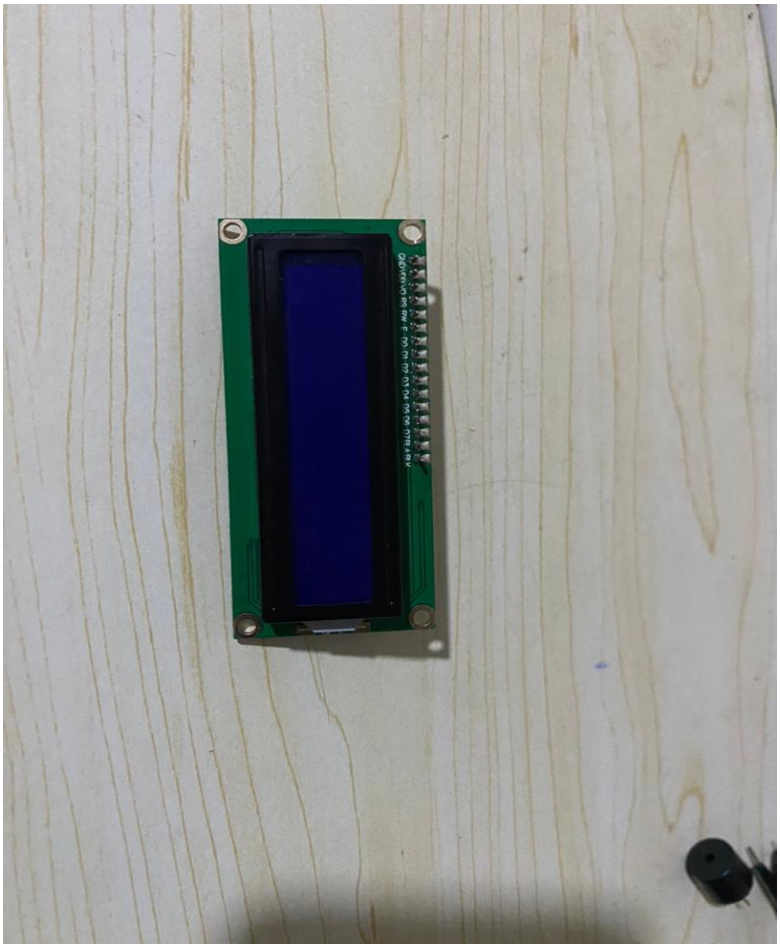
Push Buttons: These are the "senses" of your project. When you press one, it makes an electrical circuit. The microcontroller can detect the change in voltage and run a certain part of your program (like incrementing a counter or displaying a different menu option).



Reset Button: While the Uno Mini has a reset button, it's nice to have an external reset button. This lets you restart the program from the very beginning without having to remove the power. It's essentially "clearing the slate" for the microcontroller.

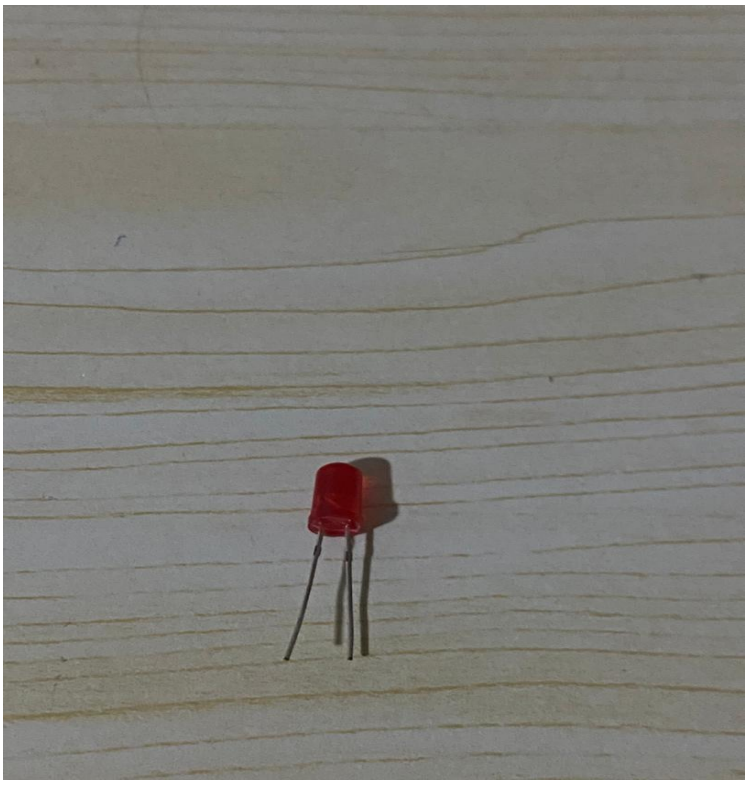
Potentiometer: This is essentially a variable resistor. This means that if you turn it, it will change resistance. In this circuit, it's used to change the brightness or "contrast" of the characters on the LCD screen.

LCD 16x2 (I2C) Display: This is your main "voice." The I2C part is very important. This uses a small backpack board, which means only 4 wires (VCC, GND, SDA, SCL) are required to talk to the Arduino, compared to the usual 10+. This can display 2 rows of 16 characters each.



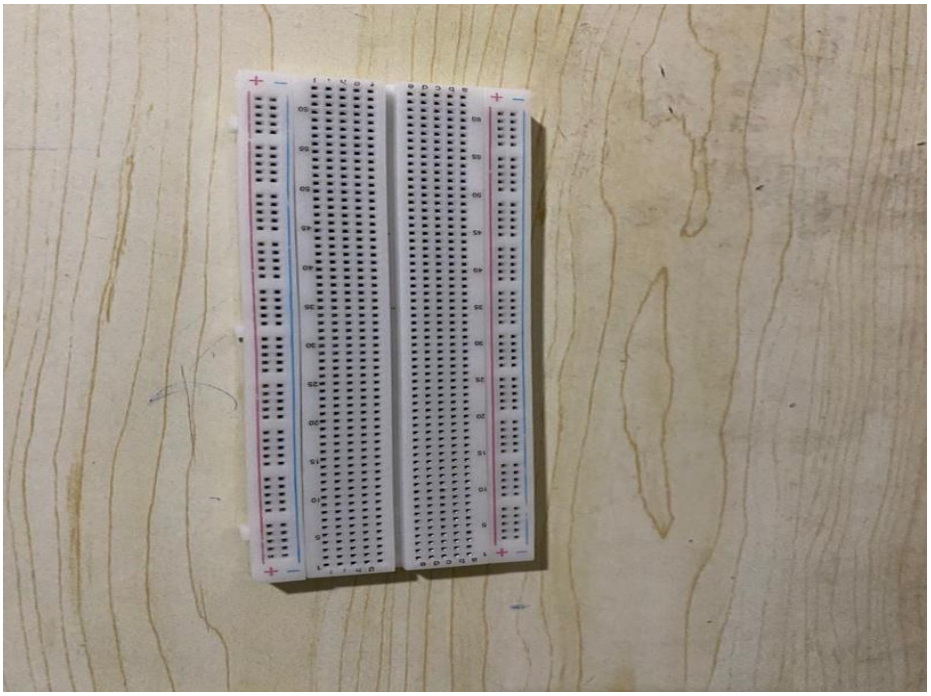
Piezo Buzzer: This is for audible feedback. The Piezo Buzzer works by the piezoelectric effect, where a ceramic disk vibrates, allowing it to beep. This is used for confirming a button press, etc

Light Emitting Diode (LED): This is a simple device for visual feedback. This is a simple binary device, meaning it can only be ON or OFF. This is normally used for showing if the system is "Ready" or if a task has been completed.



Resistor (220Ω): This is a speed limit for the electricity. You connect this in series with the LED. The Arduino sends out a voltage of 5V, but the LED only requires a voltage of 2V. The resistor "absorbs" the remaining voltage to prevent the LED from burning out.

Breadboard: This is the “construction site.” It allows you to plug things in and connect them using the metal rails without having to solder anything.



Male & Female Jumper Wires: This is the "nervous system."

Male-to-Male: This is used to connect two points on the breadboard together, or a breadboard to the Arduino.

Male-to-Female: This is used to connect the Arduino pins directly to the pins on the back of the I2C LCD or any sensor that does not have a small enough width to be inserted into the breadboard.

Female jumpers

Male jumpers

CODE OVERVIEW

1.Library Import & LCD Initialization

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(3, 4, 8, 9, 10, 11);
```

This imports the display library of the liquid crystal display and sets the data pins of the lcd to pins 3,4,6,9,10,11 of the Arduino mini.

2.Pin Definitions

```
const int patientBtn[] = {12, 13};
const int NUM_PATIENTS = 2;
const int resetBtn = 5;
const int buzzerPin = 2;
const int ledPin = A0;
```

This defines all the hardware pins used by the system and assigns them to named constants. With pins 12 and 13 being the patient call buttons, pin 5 being the nurse's reset button, pin 2 driving the buzzer and pin A0 being used as a digital output for the alert LED.

3.Alert Queuing and Display Slot

```

const int MAX_QUEUE = 10;
int displaySlot[2] = {0, 0};
bool slotActive[2] = {false, false};
unsigned long slotTime[2] = {0, 0};
int queue[MAX_QUEUE];
int queueCount = 0;

```

This portion of the code sets up the variables that keep track of which patient buttons have been pressed or have called for help and stores them in two display slots matching the two rows of the LCD screen. If both slots are full, extra alerts are held in a queue until a slot opens up.

4. Timing

```
const unsigned long ALERT_TIMEOUT = 120000UL;
const unsigned long BUZZER_DURATION = 3000UL;
unsigned long buzzerStart = 0;
bool buzzerOn = false;
```

This section of the code has to do with timing. With the help of this section of the code the system will automatically remove an unattended alert after 2 minutes and the buzzer will sound for 3 seconds each time a new alert comes in.

5. LED lighting

```
void updateLED() {
  bool anyAlert = (slotActive[0] || slotActive[1] || queueCount > 0);
  digitalWrite(ledPin, anyAlert ? HIGH : LOW);
}
```

This function controls the LED light. It checks if there are any active alerts in either display slot or waiting in the queue, and if there are, it turns the LED on. As soon as all alerts have been cleared, the LED turns off.

6. LCD update

```
void updateDisplay() {
  lcd.clear();
  if (!slotActive[0] && !slotActive[1]) {
    lcd.print("System Ready");
    lcd.setCursor(0, 1);
    lcd.print("No Active Alerts");
    return;
  }

  if (slotActive[0]) {
    lcd.setCursor(0, 0);
    lcd.print("!Pat "); lcd.print(displaySlot[0]); lcd.print(" Emergency!");
  }

  lcd.setCursor(0, 1);
  if (slotActive[1]) {
    lcd.print("-Pat "); lcd.print(displaySlot[1]); lcd.print(" Waiting...");
  } else if (queueCount > 0) {
    lcd.print("+"); lcd.print(queueCount); lcd.print(" more pending");
  }
}
```

This function updates what is shown on the LCD screen whenever the alert status changes. When there are no active alerts, it displays "System Ready" and "No Active Alerts". When a patient calls for help, it displays “-Pat (patient’s number) Emergency “. If a second patient is waiting, they appear on the second row, and if there are even more patients in the queue the screen shows how many are still pending.

7. Queue promotion

```
void promoteFromQueue() {
  if (queueCount == 0) return;
  for (int s = 0; s < 2; s++) {
    if (!slotActive[s] && queueCount > 0) {
      displaySlot[s] = queue[0];
      slotActive[s] = true;
      slotTime[s] = millis();
      for (int i = 0; i < queueCount - 1; i++) {
        queue[i] = queue[i + 1];
      }
      queueCount--;
    }
  }
}
```

This function focuses on promoting the next waiting patient from the queue onto the screen whenever a free slot appears. It works in order, so the patient who has been waiting the longest gets moved up first. Once a patient is promoted to the screen, they are removed from the queue and the remaining patients in the queue shift forward to fill the gap.

8. Adding alerts

```
void addAlert(int patient) {
  if ((slotActive[0] && displaySlot[0] == patient) ||
      (slotActive[1] && displaySlot[1] == patient)) return;
  for (int i = 0; i < queueCount; i++) if (queue[i] == patient) return;

  if (!slotActive[0]) {
    displaySlot[0] = patient; slotActive[0] = true; slotTime[0] = millis();
  } else if (!slotActive[1]) {
    displaySlot[1] = patient; slotActive[1] = true; slotTime[1] = millis();
  } else if (queueCount < MAX_QUEUE) {
    queue[queueCount++] = patient;
  }
  digitalWrite(buzzerPin, HIGH);
  buzzerOn = true;
  buzzerStart = millis();

  updateLED();
  updateDisplay();
}
```

This function runs every time a patient presses their call button. It first checks if that patient is already on the screen or in the queue and if they are already on display or in queue it ignores as to avoid registering the same issue twice .If they are not already in the system, they are placed into the first available display slot or added to the queue if both slots are full. The buzzer then beeps and the LED and screen are immediately updated to reflect the new alert.

9. Dismiss Alerts

```
void dismissAlert() {
  if (slotActive[0]) {
    if (slotActive[1]) {
      displaySlot[0] = displaySlot[1];
      slotTime[0] = slotTime[1];
      slotActive[1] = false;
    } else {
      slotActive[0] = false;
    }
    promoteFromQueue();
  }

  updateLED();
  updateDisplay();
  if (!slotActive[0] && !slotActive[1] && queueCount == 0) {
    digitalWrite(buzzerPin, LOW);
    buzzerOn = false;
  }
}
```

This function runs when the nurse presses the reset button to acknowledge an alert. It removes the top priority alert from the screen and the next priority is moved up onto the first slot of the LCD allowing a shift of other alerts in the queue, upwards.

10. Timeout and Buzzer Control

```
void checkExpiry() {
  bool changed = false;
  unsigned long now = millis();
  for (int i = 0; i < 2; i++) {
    if (slotActive[i] && (now - slotTime[i] >= ALERT_TIMEOUT)) {
      slotActive[i] = false;
      changed = true;
    }
  }
  if (changed) {
    promoteFromQueue();
    updateLED();
    updateDisplay();
  }
}

void handleBuzzer() {
  if (buzzerOn && (millis() - buzzerStart >= BUZZER_DURATION)) {
    digitalWrite(buzzerPin, LOW);
    buzzerOn = false;
  }
}
```

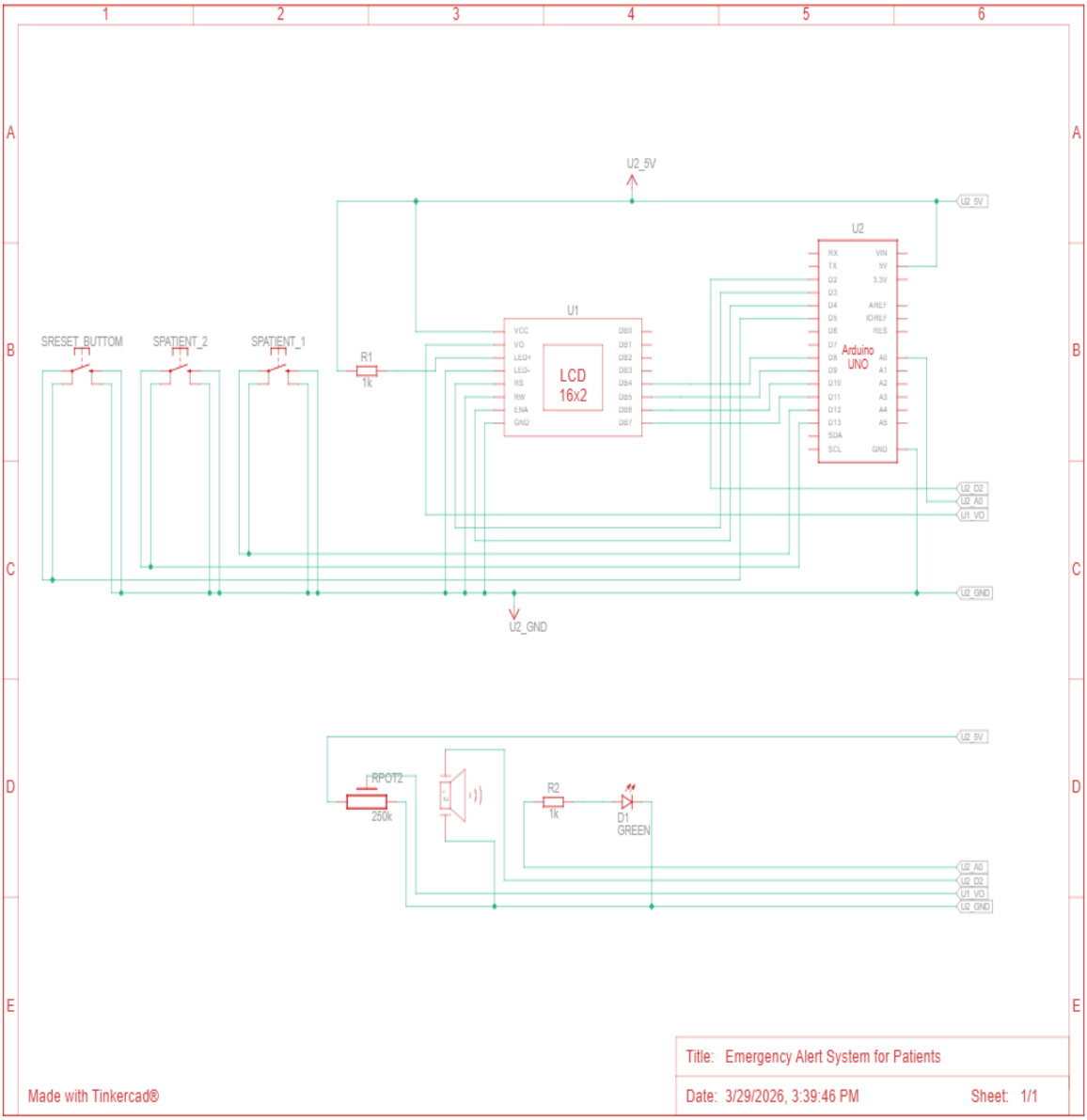
These two functions manage the timing of the system. The checkExpiry() function automatically removes any alert that has been on the screen for more than 2 minutes without a response from the nurse. The handleBuzzer() function switches the buzzer off after 3 seconds so it does not sound indefinitely.

11. The Heartbeat of the Program

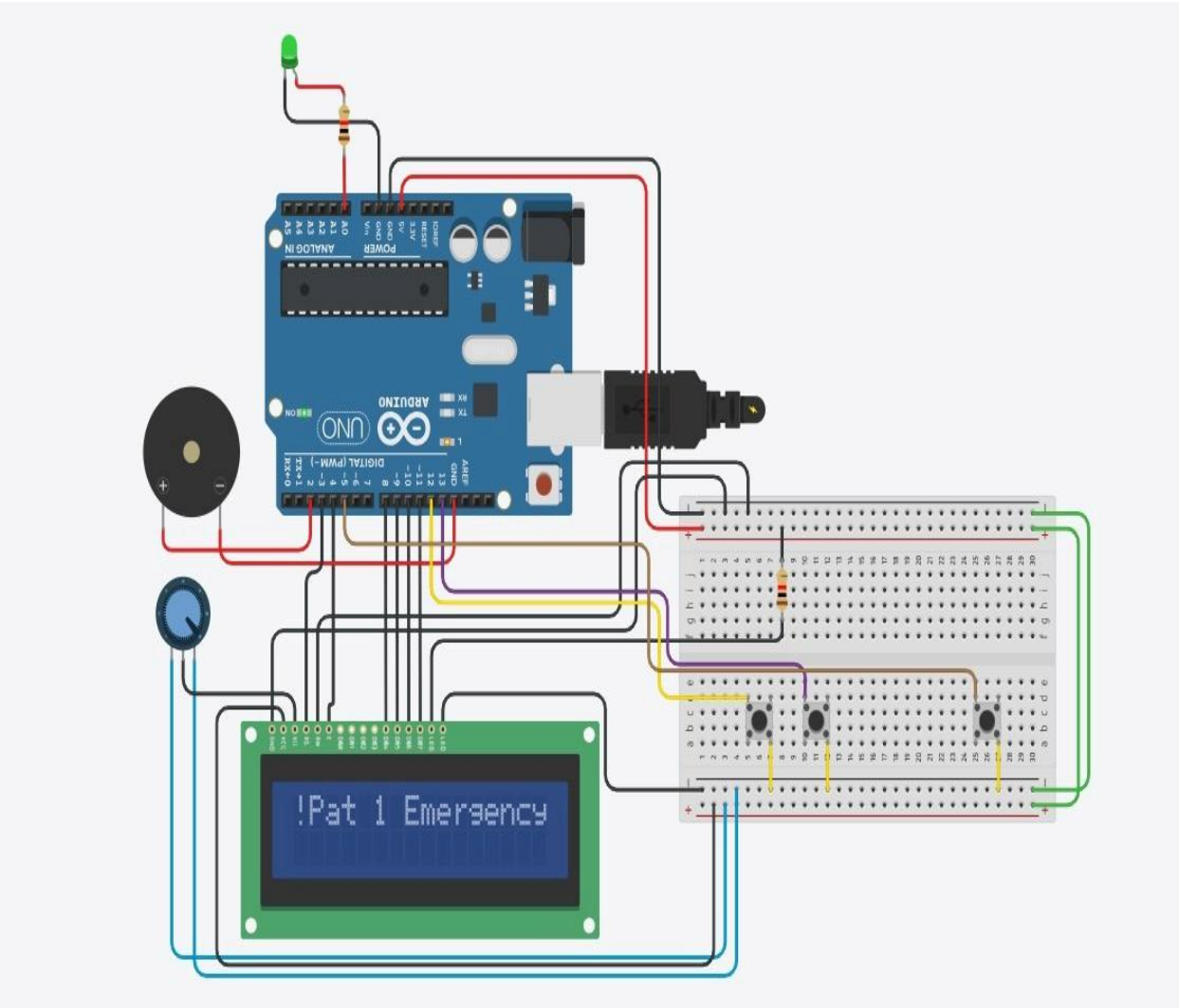
```
void setup() {
  lcd.begin(16, 2);
  for (int i = 0; i < NUM_PATIENTS; i++) pinMode(patientBtn[i], INPUT_PULLUP);
  pinMode(resetBtn, INPUT_PULLUP);
  pinMode(buzzerPin, OUTPUT);
  pinMode(ledPin, OUTPUT);
  updateLED();
  updateDisplay();
}
void loop() {
  for (int i = 0; i < NUM_PATIENTS; i++) {
    if (digitalRead(patientBtn[i]) == LOW) {
      addAlert(i + 1);
      delay(300);
    }
  }
  if (digitalRead(resetBtn) == LOW) {
    dismissAlert();
    delay(300);
  }
  handleBuzzer();
  checkExpiry();
}
```

These two functions setup() and loop () are the heartbeat of the entire program. The setup() function runs once when the system powers on, initialising the LCD screen, configuring all the buttons and output pins, and displaying the idle screen. The loop() function then runs continuously, constantly checking if any patient button or the reset button has been pressed and responding accordingly. The delay(300) after each button press acts as a debounce to prevent a single press from being registered multiple times. At the end of every loop cycle handleBuzzer() and checkExpiry() are called to keep the timing functions running in the background.

CIRCUIT SCHEMATIC



PICTURE OF THE CIRCUIT SCHEMATIC



PICTURE OF THE EMERGENCY ALARM SYSTEM FOR PATIENTS

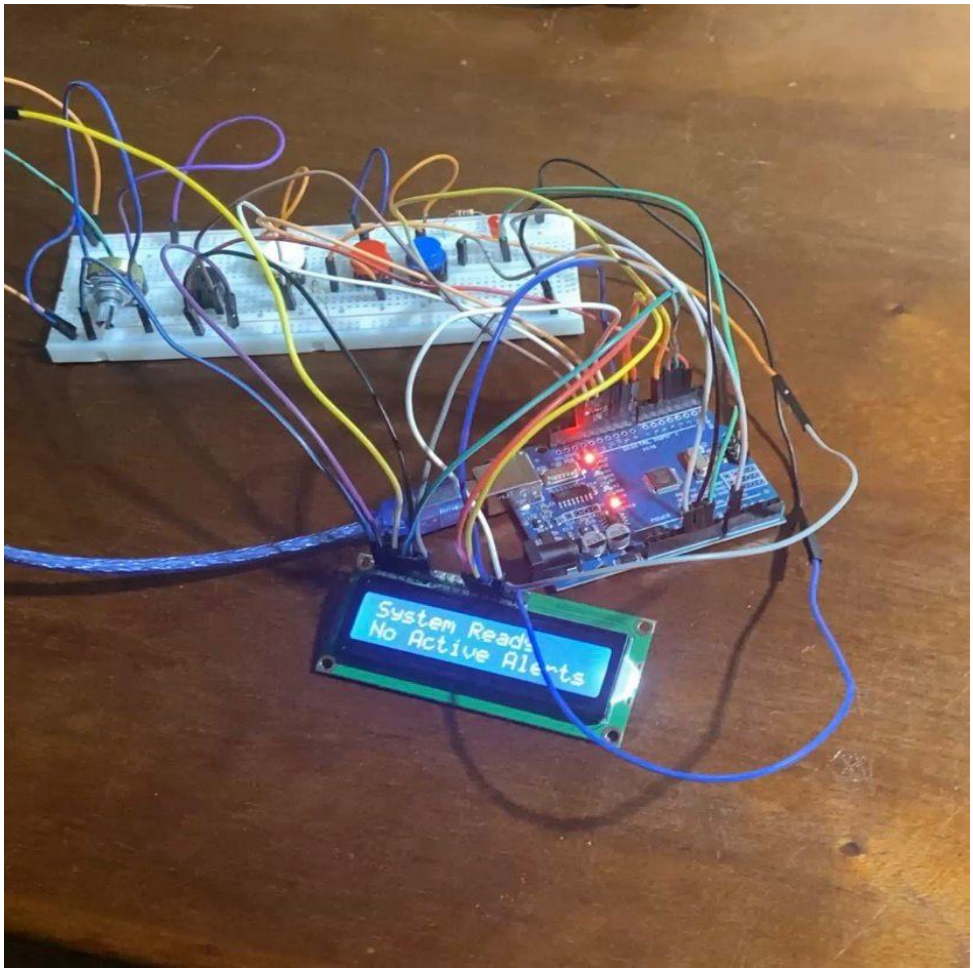
DEMONSTRTATION

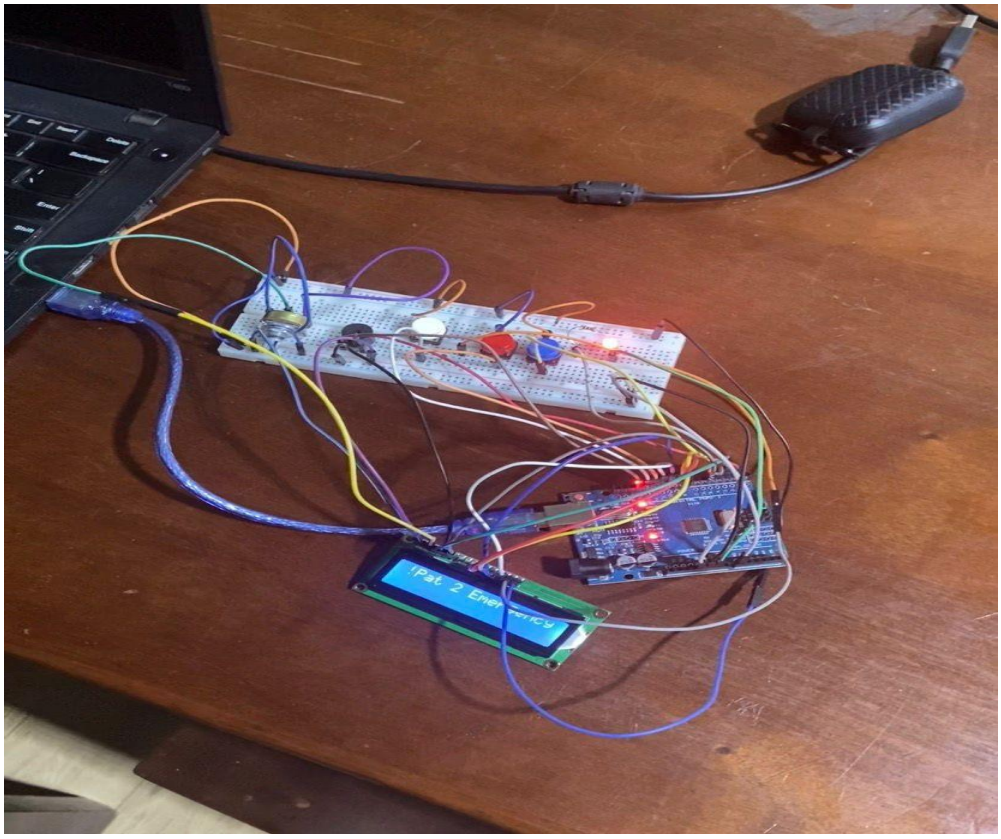
The System Demonstration and Hardware Integration project entails creating a patient alert system, which has various states and hardware. The states in the patient alert system are System Ready, Single Patient Alert, Stacked Alerts, and System Reset.

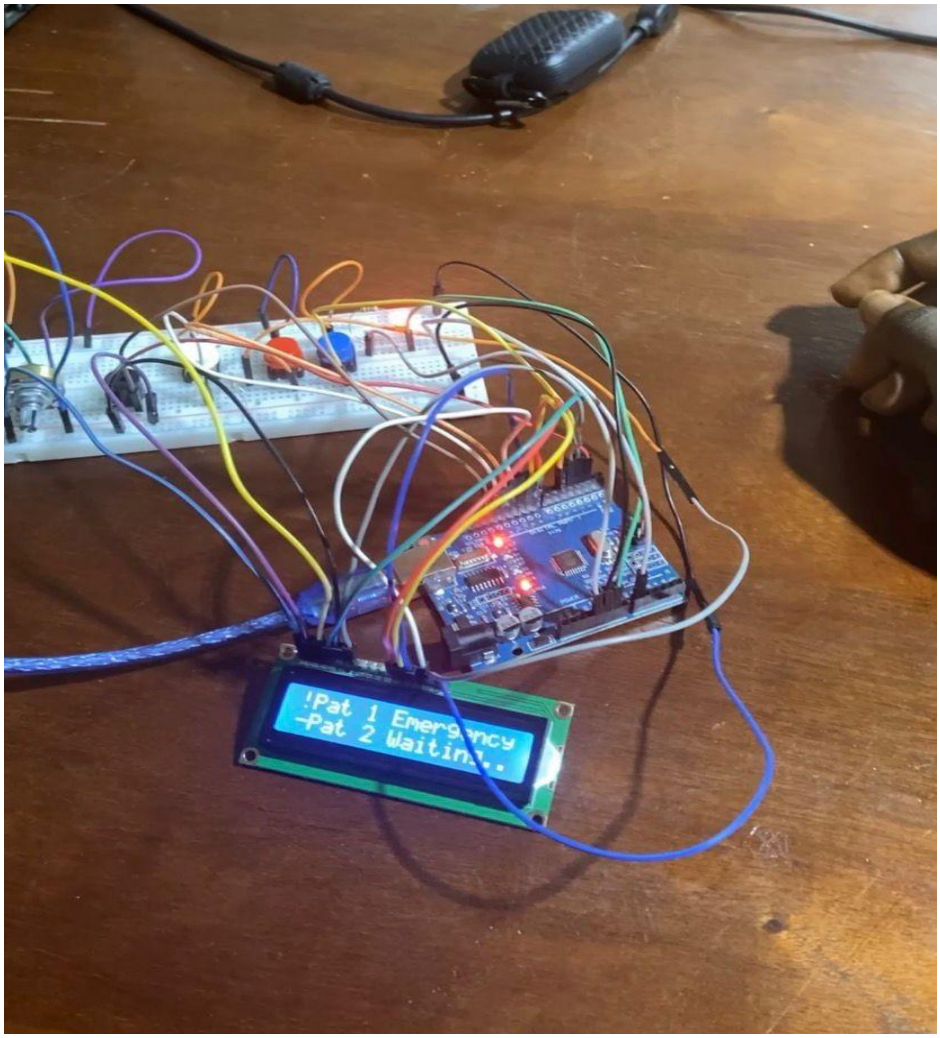
In the System Ready state, the system is initialized, and the LCD shows "System Ready." The LED and buzzer are not activated. However, when a patient sends an alert, for example, "Patient 1," the system moves to the Single Patient Alert state, where the LCD shows an emergency message, the LED is activated, and the buzzer beeps for 5 seconds. When another patient, for example, "Patient 2," sends an alert, the system moves to the Stacked Alerts state, where priority is managed on the LCD and the buzzer is re-triggered.

In terms of hardware, the patient alert system has an Arduino Uno microcontroller, a 16x2 LCD display, a potentiometer for LCD contrast, a piezo buzzer, and a status LED. Three tactile switches are used for user input for "Patient 1 Call," "Patient 2 Call," and "System Reset." The patient alert system has pull-down resistors for logic transition stability and current-limiting resistors for the LED.

In terms of programming, the patient alert system is programmed in C++ and is implemented in the Arduino Framework, using the Liquid Crystal library. The patient alert system is validated through two stages: Tinkercad digital simulation and prototyping.

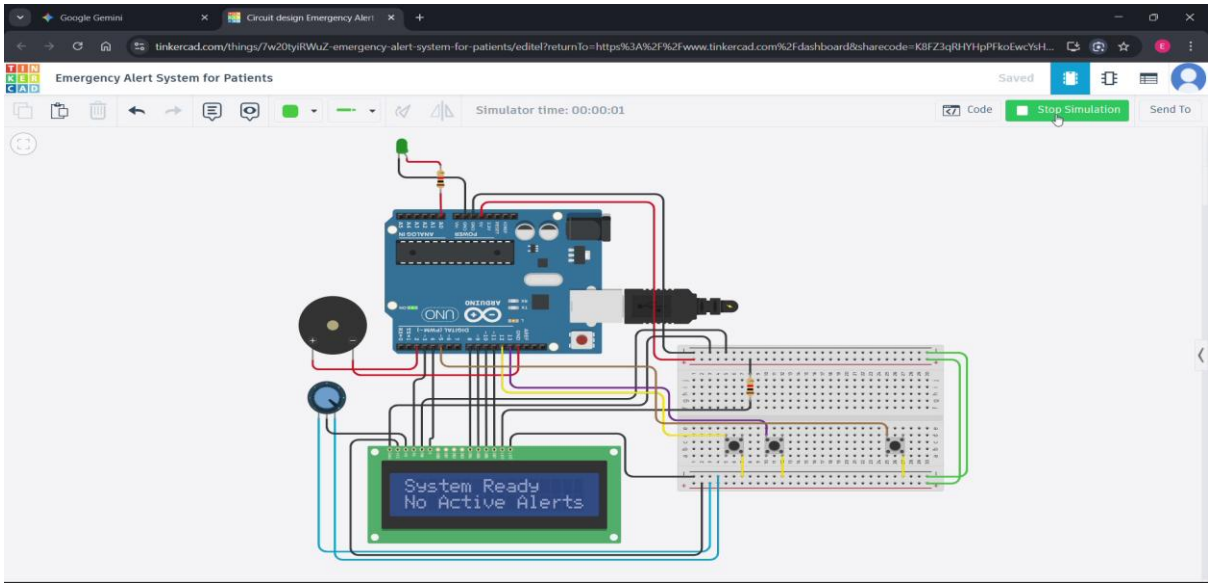

System Ready
No Active Alerts


Pat 2 Emergency



PICTURES OF THE EXPERIMENT IN ACTION

SIMULATION



Future Roadmap: Scalability & Innovation

1. Hybrid Wireless Connectivity
IoT (Wi-Fi): This will allow campus-wide monitoring, where the server can monitor alerts from different wards or buildings without distance constraints.
Bluetooth (BLE): This will allow for low power, high-speed communication for instant "Room-to-Reception" alerts over shorter distances.

2. Infrastructure Optimization
Cost Reduction: This will eliminate the need for costly cabling, which would have meant significant labor costs for routing the wires through the hospital walls.
Seamless Maintenance: Moving over to "Plug-and-Play" wireless nodes will reduce system down time during maintenance or upgrade

activities.

3. Human Machine Interface (HMI) Evolution Capacitive Touch: This will replace traditional buttons with touch-sensitive pads, which will reduce wear and tear on the system. Clinical Hygiene: Touch-sensitive surfaces are easier to sanitize, which will directly contribute to hospital hygiene, infection control, and sterilization.

Challenges Encountered

Several challenges were encountered in the simulation and physical testing phases. All challenges were systematically identified and resolved:

1: LCD screen did not display anything in the simulation using Tinkercad. The code was correct.

Reason for challenge: The LCD's RW pin was not connected to GND.

Solution for challenge: Connected the RW pin to GND.

2: The buzzer sounded continuously since the simulation started, rather than only sounding when a button was pressed.

Reason for challenge: The buzzer's pin was set to be a digital output but without being set to LOW.

Solution for challenge:Added digitalWrite(buzzerPin, LOW) and noTone(buzzerPin) as the first two lines of the setup().

3: Some button presses were skipped when the buzzer or LCD was being updated.

Reason for challenge: The delay() function was used in the code for the alerts and buzzer sounds. This caused the program to be stuck and unable to check for button presses.

Solution for challenge:Replaced delay() with the use of the millis() function.

4: The LCD screen did not display any text even when the wiring was correct.

Reason for challenge: The potentiometer's contrast was not turned.

Solution for challenge: Turned the potentiometer's knob until the text became clear.

5: The LCD screen did not display any text when using the I2C LCD with a different circuit configuration.

Reason for challenge: The I2C address used was incorrect.

Solution for challenge: The code used address 0x27, but the Tinkercad I2C LCD component used address 0x20.

CONCLUSION

Summary of Findings

The Emergency Alert System for Patients was successfully designed, built, and tested. The system meets all originally defined requirements: patients can call for help using a push button, the LCD immediately displays the patient number and emergency status, the buzzer sounds

for 3 seconds, and the LED remains lit until all alerts are cleared by nursing staff.

The two-slot display system with queue management proved effective at handling multiple simultaneous alerts without losing any calls. The non-blocking millis()-based timing architecture ensured the system remained fully responsive at all times, never missing a button press regardless of what other operations were in progress.

Our Emergency Alert System effectively illustrates the possibilities of focused Electrical Engineering and Embedded Programming to bridge critical gaps in the field of healthcare communications. Through optimizing our code with C++ and developing a sophisticated Priority Logic, we have created a system that is both reliable and scalable. We firmly believe that with the integration of the IoT and Touch Interface enhancements we have proposed, this system has the potential to become a cost-effective, life-saving standard for modern medical wards. Thank you for your time. I'm now open to any technical questions.